Multi-Modal Clinical Synthetic Data (Structured + Clinical Notes + Vitals Time-Series)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The outcome . . . 4

The approach . . . 4

Why it's safe . . . 4

Bottom line . . . 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

The deployment model — Assess → Design → Generate/Build → Validate → Expand . . . 6

Benefits . . . 6

Limitations (read these) . . . 6

Requirements 7

R1. A working structured (tabular) synthetic backbone . . . 7

R2. A per-patient clinical state contract (the shared source of truth) . . . 7

R3. Note types, clinical template/phrase references (NLP-ready, no real PHI) . . . 8

R4. Vitals/time-series channels, sampling and physiologic ranges . . . 8

R5. Cross-modal consistency rules and pass/fail gates . . . 8

R6. A synthetic-safety / no-real-PHI control for the multi-modal set . . . 9

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

1. A Python data/engineering environment . . . 9

2. A target clinical data model / standard . . . 10

3. An isolated non-production test environment . . . 10

4. Privacy, governance & compliance stakeholders and policy . . . 10

5. AI/QA use cases and a pilot scope . . . 10

6. Reference/real-data access for fidelity (read-only, minimized) . . . 11

Implementation 11

Stage 1 — Assess . . . 11

Stage 2 — Design . . . 12

Stage 3 — Generate/Build (pilot cohort) . . . 12

Stage 4 — Validate (pass/fail gates) . . . 13

Stage 5 — Expand . . . 13

Testing & Validation 14

T1 — Modality coverage (every modality present per patient) . . . 14

T2 — Fidelity vs. reference . . . 14

T3 — Physiologic plausibility of vitals . . . 14

T4 — Cross-modal consistency (the modalities tell the same story) . . . 15
T5 — NLP-readiness of notes . . . 15
T6 — Synthetic-safety / no-real-PHI scan (release blocker) . . . 15
Release decision . . . 16
Troubleshooting 16
The note and the vitals contradict each other (T4 consistency FAIL) . . . 16
A note states a diagnosis that isn't in the structured problem list (T4 consistency FAIL) . . . 16
Vitals out of physiologic range (T3 plausibility FAIL) . . . 16
`scope_config_helper.py validate` reports FAIL . . . 17
A modality is missing for some patients (T1 coverage FAIL) . . . 17
Fidelity shows "NOT-SCORED" . . . 17
Fidelity FAIL (distance over threshold) . . . 17
Synthetic-safety scan hit — a real-PHI pattern (T6 FAIL — hard block) . . . 17
Notes are empty or malformed (T5 NLP-readiness FAIL) . . . 18
Vendor synthetic-data / AI framework behaves differently than documented here . . . 18
Best Practices 18
Generate from one shared state — never author modalities independently . . . 18
Make "consistent" measurable, and gate on it . . . 18
Keep templates synthetic and version-controlled . . . 19
Anchor vitals to physiology and to the note narrative . . . 19
Validate against a reference, and state the reference . . . 19
Run the no-real-PHI scan on every set — free-text is the biggest risk surface . . . 19
Pilot-first, expand only after the gate passes . . . 19
Keep the config the single driver; keep everything reproducible . . . 19
Stay inside the boundary . . . 19
Deliverables Summary 19
Documentation . . . 20
The 5 Python tools . . . 20
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary

The outcome

Clinical AI runs on more than tables. Document models read **clinical notes**, signal and sequence models read **vitals and time-series**, and multi-modal pipelines depend on those modalities *agreeing* with one another. Synthetic data that stops at structured tables can only test a fraction of what real care actually uses. This solution extends synthetic generation to **multi-modal clinical data** — structured records **plus** realistic, NLP-ready clinical notes **plus** vitals/time-series — kept **cross-modally consistent for a single synthetic patient**, so AI teams can test document-, signal-, and sequence-based models, not just tabular ones.

The approach

The generator is driven by one **per-patient clinical state** — the structured record plus a small set of derived clinical facts — that all three modalities render from. A template/pattern **note synthesizer** produces labelable free-text, a **vitals generator** produces physiologically plausible signals, and both read the same state, so the note narrative, the structured record, and the vitals streams tell the **same story**. A **cross-modal consistency validator** then proves it — diagnoses in notes appear in the structured problem list, asserted vitals events have matching signals, and everything stays in plausible ranges. Delivery follows a **pilot-first Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model with explicit pass/fail gates and a clean rollback, and ships five safe-by-default **Python** tools that automate assessment, generation, fidelity/coverage reporting, consistency/safety validation, and config.

Why it's safe

Every artifact is **synthetic**. Notes are slot-filled from templates and phrase banks — never copied from real patient text — and **no real PHI is ever written into the shared harness**. All work runs in an **isolated non-production** environment; any real reference data used for fidelity stays **read-only and minimized**. A **release-blocking no-real-PHI scan** runs on every set, with free-text notes treated as the primary risk surface. The solution provides the technical **method and evidence** only — HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, and disclosure-risk acceptance are **deferred to the organization's privacy, compliance, and legal functions**. Vendor synthetic-data and AI frameworks are integrated at the pattern level, with version-specific behavior verified against the vendor's documentation.

Bottom line

Synthetic clinical data across modalities — structured, notes, and vitals/time-series, consistent per patient — so AI teams can test the models real care actually uses, safely, at a fraction of bespoke consulting cost.

Solution Overview

What this is

Most synthetic clinical data stops at **tables** — patients, encounters, conditions, medications, labs. But real clinical AI runs on more than tables. Document models read **clinical notes**, signal and sequence models read **vitals and time-series**, and the interesting failure modes live in how those modalities *relate* to one another. A discharge summary that says "afebrile, hemodynamically stable" is wrong if the paired vitals stream shows a fever spike and a crashing blood pressure. A tabular record that codes sepsis but a note that never mentions it teaches an NLP model the wrong thing.

This solution extends synthetic generation **beyond structured tables to multi-modal clinical data**:

- **Structured records** — the tabular backbone (demographics, encounters, conditions, medications, labs) generated against a target model.
- **Clinical notes** — realistic, NLP-ready free-text (admission notes, progress notes, discharge summaries) produced **template- and pattern-driven** from the structured record, so the narrative is *derived from* the same synthetic patient, not written independently.
- **Vitals / time-series** — heart rate, blood pressure, respiratory rate, temperature, SpO■ and similar signals as time-ordered series whose shape reflects the patient's synthetic condition and trajectory.

The defining property is **cross-modal consistency**: for a single synthetic patient, the structured record, the notes, and the vitals **tell the same clinical story**. Every artifact is **synthetic** — no real Protected Health Information (PHI) is used to author the notes or signals, and nothing real is written into the shared harness.

How it works

The generator is driven by a **per-patient clinical state** (the target model record plus a small set of derived clinical facts — e.g. "febrile", "hypotensive episode at hour 14", "on vasopressors"). That single state is the shared source of truth that all three modalities render from:

1. **Structured** — the tabular record in the target model (see below).
2. **Notes** — a template/pattern note synthesizer fills clinically-worded templates from the state's facts. Templates are pattern-based (slot-filling with controlled vocabularies and phrase banks), **not** copied from real notes and **not** free-form generation of unverifiable text.
1. **Vitals** — a signal generator produces time-series whose baselines, trends, and events are keyed to the same facts (a documented fever produces a matching temperature curve).

Because all three read the **same** state, a **cross-modal consistency validator** can then check that they agree (the note's stated diagnoses appear in the structured problem list; a note that asserts a vitals event has a matching signal; units and ranges are physiologically plausible). Disagreements are reported as consistency defects to fix before the dataset is released to AI/QA teams.

Target model (structured backbone)

Generate the structured layer against a defined **clinical data model / interoperability standard** — typically **FHIR** (resources: `Patient`, `Encounter`, `Condition`, `MedicationRequest`, `Observation`) or **OMOP CDM** (person, visit_occurrence, condition_occurrence, drug_exposure, measurement) — or the source system's own schema. **Notes** attach as document/`DocumentReference`-style records and **vitals** as time-stamped `Observation`/measurement rows, so the multi-modal output stays inside the same

target model rather than becoming three disconnected exports. Confirm the exact target model and in-scope domains with the AI/QA teams before generating (see `02-requirements.md`).

The deployment model — Assess → Design → Generate/Build → Validate → Expand

Every solution in this category follows the same **pilot-first** model with explicit pass/fail gates and a documented rollback:

• **Assess** — profile which modalities the AI/QA use cases actually need (notes? vitals? both?), confirm the target model, the isolated environment, and the governing privacy policy. Produce the **Modality Scope – Pilot – v1** record.

• **Design** — decide the per-patient clinical state model, the note types and template families, the vitals channels and sampling, and the consistency rules. Produce the **Multi-Modal Design – Pilot – v1**.

• **Generate/Build** — generate the structured record, synthesize notes, generate vitals for a **pilot cohort**, all from the shared state. Produce the** Multi-Modal Synthetic Set – Pilot – v1**.

• **Validate** — run the fidelity/coverage report and the cross-modal consistency validator against pass/fail **gates** (modalities present, cross-modally consistent, physiologically plausible, privacy checks clean). Produce the **Multi-Modal Validation Report – Pilot – v1**.

• **Expand** — only after the pilot passes, widen the cohort, add note types / vitals channels, and scale — re-validating at each step. Rollback = discard the failing synthetic set (it is synthetic and isolated) and regenerate; nothing production or real-PHI is touched.

Benefits

• **Test the models real care uses** — document, signal, and sequence models, not just tabular ones.

• **Cross-modally consistent** — a single synthetic patient's records, notes, and vitals agree, so multi-modal pipelines (e.g. note + vitals fusion) can be exercised end to end.

• **NLP-ready notes without real PHI** — template/pattern-driven text that is labelable and safe to share in the harness.

• **Safe by construction** — synthetic-only, isolated non-production, safe-by-default tooling.

Limitations (read these)

• **Synthetic, not real.** Template/pattern notes and modeled vitals approximate clinical language and physiology; they are for **testing and development**, not clinical inference or evidence.

• **Fidelity is bounded by the reference.** Realism claims hold only against the reference statistics or distributions you validate against (global requirement §6).

• **Not a de-identification method.** This generates synthetic data; it does **not** de-identify real

data, and it makes **no** HIPAA/HITRUST determination. Those determinations, IRB/privacy sign-off, and disclosure-risk acceptance are **deferred to the organization's privacy, compliance, and legal functions**.

- **Vendor tools at the pattern level.** Where a synthetic-data platform or AI framework is used

(e.g. Synthea-style generators, commercial platforms), integrate at the capability/pattern level and **verify version-specific behavior against the vendor's documentation** — this solution does not reproduce vendor console internals.

Requirements

These are the **solution-specific** prerequisites for extending synthetic generation to multi-modal clinical data (structured + notes + vitals/time-series). They build **on top of** the six category-wide requirements in `global-requirements-synthetic.md` — a Python data environment (§1), a target clinical data model/standard (§2), an isolated non-production test environment (§3), privacy/governance stakeholders and policy (§4), AI/QA use cases and a pilot scope (§5), and read-only/minimized reference access for fidelity (§6). **Verify the global six first**; do not duplicate them. The items below are the additional prerequisites this multi-modal solution needs, each with how to *verify* it and how to *implement* it if missing.

> ***Boundary reminder.*** *Everything below operates on* ***synthetic*** *data in the* ***isolated non-production*** *environment. Never place real PHI into the shared harness. Regulatory/legal determinations are deferred to privacy/compliance/legal.*

R1. A working structured (tabular) synthetic backbone

- **Description:** An existing, validated **structured** synthetic generator/harness producing patients,

encounters, conditions, medications and labs in the target model — the tabular layer this solution extends. (If you do not yet have one, that is a separate structured-generation effort; this solution assumes the tabular backbone exists or is stood up first.)

- **Why required:** Notes and vitals are **derived from** the structured record's per-patient clinical

state. Without a trustworthy structured backbone there is nothing consistent to narrate or to signal.

- **How to verify:** Confirm you can generate a small structured cohort in the target model and that it

passes its own basic validation (schema-valid, plausible field values).

- **How to implement if missing:** Stand up the structured backbone (in-house generator or a

Synthea-style/commercial platform integrated at the pattern level — verify version specifics against the vendor's documentation) and validate it before adding modalities.

R2. A per-patient clinical state contract (the shared source of truth)

- **Description:** An agreed, documented **clinical state** object per synthetic patient — the structured

record plus a small set of derived clinical facts (e.g. `febrile`, `hypotensive_episode`, `on_vasopressors`, primary/secondary diagnoses, admission→discharge trajectory) that **both** the note synthesizer and the vitals generator read.

• **Why required:** Cross-modal consistency is only achievable if all modalities render from **one** shared state. Independently authored notes and vitals will drift and contradict each other.

• **How to verify:** Confirm a written schema for the clinical state and that a sample structured record can be mapped to it (each downstream fact has a defined source field/rule).

• **How to implement if missing:** Define the clinical-state schema (with the AI/QA and clinical SMEs) and a deterministic mapping from the structured record to the derived facts before generating notes or vitals.

R3. Note types, clinical template/phrase references (NLP-ready, no real PHI)

• **Description:** The **note types** in scope (e.g. admission note, progress note, discharge summary) and the **templates / controlled vocabularies / phrase banks** used to synthesize them — authored or curated in-house, or drawn from **published, non-PHI** clinical templating references. Templates are slot-filling patterns, **not** copies of real patient notes.

• **Why required:** Notes must be realistic and labelable for NLP testing while remaining **synthetic**. Copying real notes would reintroduce PHI and defeat the purpose.

• **How to verify:** Confirm the note-type list, that each has a template family with slots keyed to clinical-state facts, and that no template contains real patient text.

• **How to implement if missing:** Author the templates and phrase banks (clinical SME review recommended) with slots mapped to R2's clinical state; keep them under version control.

R4. Vitals/time-series channels, sampling and physiologic ranges

• **Description:** The **vitals channels** in scope (e.g. HR, systolic/diastolic BP, respiratory rate, temperature, SpO■), their **sampling cadence** (e.g. hourly), duration (encounter length), and **physiologically plausible ranges/trends** per clinical state (baseline, expected events).

• **Why required:** The signal generator needs defined channels, cadence, and plausible ranges to produce time-series that (a) are realistic and (b) match the same patient's notes and structured record.

• **How to verify:** Confirm the channel list, cadence, and agreed normal/abnormal ranges (from the reference in global §6 or published reference ranges), and that abnormal events map to R2 facts.

• **How to implement if missing:** Define the channels, cadence, and range/trend rules per clinical state; document the reference for the ranges before generating.

R5. Cross-modal consistency rules and pass/fail gates

• **Description:** The explicit **consistency rules** that bind the modalities (e.g. diagnoses stated in a note must appear in the structured problem list; a note asserting a vitals event must have a matching signal event; units and ranges must be physiologically valid) and the **pass/fail gates** for release (modalities present, cross-modally consistent, physiologically plausible, privacy checks clean).

• **Why required:** "Consistent" must be *measurable* to gate a release. Without codified rules the validator cannot pass or fail a dataset objectively.

• **How to verify:** Confirm a written rule set and the numeric/boolean thresholds each gate requires.

• **How to implement if missing:** Author the consistency rules and gate thresholds with the AI/QA and privacy stakeholders before the Validate stage.

R6. A synthetic-safety / no-real-PHI control for the multi-modal set

• **Description:** A control ensuring the notes and vitals contain **no real PHI** and that nothing real is written to the shared harness — e.g. an allow-list of synthetic identifiers, a scan for real-name / MRN / date patterns, and confirmation that any real reference data used for ranges stayed read-only and minimized (global §6).

• **Why required:** Notes and free-text especially are a re-identification risk surface; this is the operational guardrail that keeps the multi-modal set safe to share in the harness.

• **How to verify:** Confirm the scan/allow-list exists and runs as part of validation, and that reference-data handling is documented as read-only/minimized.

• **How to implement if missing:** Implement the synthetic-safety scan and reference-handling record before releasing any multi-modal set to AI/QA teams. (Note: this is a **technical guardrail**, not a de-identification determination — that remains with privacy/compliance/legal.)

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate **synthetic** (artificial) data and run in **isolated non-production** test environments. **Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment** — where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls. **Synthetic-data and digital-twin tools** (e.g. Synthea, and commercial platforms) and **AI/model frameworks** are integrated at the pattern/architectural level; specific vendor console internals are **cited or deferred** to that vendor's documentation, never invented. **Regulatory and legal interpretation** (HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is **deferred to the organization's privacy, compliance, and legal functions** — the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain

(data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

Follow these steps in order to extend synthetic generation to multi-modal clinical data, **pilot-first**, following **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** with explicit pass/fail gates and a documented rollback. Every artifact is **named** with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in later steps, testing, and the scripts. All work runs in the **isolated non-production** environment on **synthetic** data — never real PHI in the shared harness.

The five Python tools referenced below ship with this solution (see `08-deliverables-summary.md`): `modality_scope_assess.py`, `multimodal_generate.py`, `fidelity_coverage_report.py`, `crossmodal_consistency_check.py`, and `scope_config_helper.py`. Each is **safe by default** (dry-run where it writes).

Stage 1 — Assess

Step 1.1 — Confirm the global prerequisites and boundary

Verify the six global requirements (`global-requirements-synthetic.md`) and the solution-specific prerequisites (`02-requirements.md`), especially R1 (structured backbone) and the isolated environment. Record the governing privacy policy and the deferral of regulatory determinations to

privacy/compliance/legal.

Step 1.2 — Profile the required modalities

Run the assessment tool against your AI/QA use cases to determine which modalities are actually needed and whether the structured backbone can support them:

```
python modality_scope_assess.py --use-cases use-cases.json
```

Capture the output as **Modality Scope – Pilot – v1** (the scope record). It states, per use case, which of {structured, notes, vitals} are in scope, the target model, note types, and vitals channels.

Gate A (Assess): target model confirmed, isolated environment confirmed, privacy policy recorded, and at least the structured backbone present. If any fails, stop and remediate per `02-requirements.md`.

Stage 2 — Design

Step 2.1 — Define the per-patient clinical state contract

From R2, write the **clinical state** schema — the structured record plus the derived facts (e.g. `febrile`, `hypotensive_episode@h14`, `on_vasopressors`, diagnoses, trajectory) — and the deterministic mapping from a structured record to those facts. Record this as **Clinical State Contract – Pilot – v1**.

Step 2.2 — Generate the run configuration

Use the config helper to produce a validated, version-controlled run config that encodes the scope, note types, vitals channels/cadence/ranges, consistency rules, and gate thresholds:

```
python scope_config_helper.py init --scope "Modality Scope – Pilot – v1" --out multimodal-config.json
python scope_config_helper.py validate --config multimodal-config.json
```

Record the validated config as **Multi-Modal Config – Pilot – v1**. This single config drives generation and validation so all stages agree.

Step 2.3 — Finalize note templates, vitals rules, and consistency rules

Confirm R3 (note templates/phrase banks, no real PHI), R4 (vitals channels/cadence/ranges), and R5 (consistency rules + gate thresholds) are captured in the config. Record the combined design as **Multi-Modal Design – Pilot – v1**.

Gate B (Design): clinical state contract written, config validates, and every downstream fact (note slot, vitals event, consistency rule) traces to a state field. If not, complete the design before generating.

Stage 3 — Generate/Build (pilot cohort)

Step 3.1 — Generate the pilot multi-modal set (dry-run first)

Generate structured + notes + vitals for a small **pilot cohort**, all rendered from the shared clinical state. The tool is **dry-run by default** — preview first, then opt in to write to the isolated harness:

```
# Preview (no files written)
python multimodal_generate.py --config multimodal-config.json --patients 25

# Write to the isolated harness location (explicit opt-in)
python multimodal_generate.py --config multimodal-config.json --patients 25 \
    --out ./harness/pilot --write
```

The output is **Multi-Modal Synthetic Set – Pilot – v1**: for each synthetic patient, a structured record (target model), one or more notes per the note types, and vitals time-series per channel — every modality derived from the same state.

Gate C (Generate): all in-scope modalities are present for every pilot patient (no patient missing notes or vitals). Use the coverage portion of the fidelity report (Step 4.1) to confirm.

Stage 4 — Validate (pass/fail gates)

Step 4.1 — Fidelity & coverage report

Measure modality coverage and fidelity of the pilot set against the reference (global §6):

```
python fidelity_coverage_report.py --set ./harness/pilot --config multimodal-config.json \
    --reference reference-stats.json --report fidelity-coverage.json
```

Record as **Multi-Modal Validation Report – Pilot – v1 (Fidelity/Coverage)**. Gate on: every modality present per patient (coverage) and fidelity metrics within the configured thresholds.

Step 4.2 — Cross-modal consistency & synthetic-safety check

Run the consistency validator — it checks that the modalities tell the same story and runs the no-real-PHI synthetic-safety scan (R5, R6):

```
python crossmodal_consistency_check.py --set ./harness/pilot --config multimodal-config.json \
    --report consistency.json
```

Record as **Multi-Modal Validation Report – Pilot – v1 (Consistency/Safety)**.

Gate D (Validate) — the release gate. The pilot set passes only if **all** hold:

1. **Modalities present** — coverage complete (Step 4.1).
2. **Cross-modally consistent** — no unresolved consistency defects (Step 4.2): note diagnoses appear in

the structured problem list; asserted vitals events have matching signals; units/ranges valid.

1. **Physiologically plausible** — vitals within configured ranges/trends (Step 4.1/4.2).
2. **Synthetic-safety clean** — the no-real-PHI scan reports no hits; reference data handling documented

read-only/minimized (Step 4.2).

Rollback: if Gate D fails, **discard** the failing synthetic set (it is synthetic and isolated) and regenerate after fixing config/templates/rules. Nothing production or real-PHI is touched, so rollback is simply deleting the pilot output and re-running Stage 3.

Stage 5 — Expand

Only after **Gate D passes** on the pilot:

Step 5.1 — Widen the cohort and modalities

Increase the patient count and add note types / vitals channels in the config, then regenerate and **re-validate at each step** (repeat Stage 4). Name expanded artifacts by scope, e.g. **Multi-Modal**

Synthetic Set – Cardiology Cohort – v1.

Step 5.2 — Hand off to AI/QA

Release the validated multi-modal set to the AI/QA teams for document-, signal-, and sequence-model testing, with the validation reports attached as evidence. Keep the config and reports under version control so any set is reproducible and its gates auditable.

Testing & Validation

These procedures validate a **Multi-Modal Synthetic Set – Pilot – v1** before it is released to AI/QA teams. Each test states its purpose, how to run it, and the **expected pass/fail**. Together they implement **Gate D** from `03-implementation.md` (the release gate). All tests run in the isolated non-production environment on synthetic data.

Run the fidelity/coverage report and the consistency/safety check on every set; do not release a set that fails any gate.

T1 — Modality coverage (every modality present per patient)

- **Purpose:** Confirm each in-scope modality (structured, notes, vitals) exists for **every** synthetic patient — no patient missing a note or a vitals stream.
- **How to run:**

```
python fidelity_coverage_report.py --set ./harness/pilot --config multimodal-config.json \
    --reference reference-stats.json --report fidelity-coverage.json
```

Inspect the `coverage` section.

- **Expected — PASS:** coverage = 100% for every in-scope modality across the cohort.
- **FAIL:** any patient missing an in-scope modality → regenerate (Stage 3) after fixing the config/state mapping.

T2 — Fidelity vs. reference

- **Purpose:** Confirm structured fields and vitals distributions are realistic **against the reference** (global §6) within configured thresholds.
- **How to run:** same command as T1; inspect the `fidelity` section (per-field/channel distances vs. thresholds).
- **Expected — PASS:** all fidelity metrics within their configured thresholds.
- **FAIL:** metrics out of threshold → adjust generation parameters/ranges and regenerate. (Fidelity claims hold only against the validated reference.)

T3 — Physiologic plausibility of vitals

- **Purpose:** Confirm vitals time-series stay within physiologically plausible ranges and follow the

expected trend/events for each patient's clinical state (R4).

- **How to run:**

```
python crossmodal_consistency_check.py --set ./harness/pilot --config multimodal-config.json \
    --report consistency.json
```

Inspect the `plausibility` section.

- **Expected — PASS:** no out-of-range or physiologically implausible values; documented events (e.g. a fever) appear in the matching channel.
- **FAIL:** out-of-range values or missing/contradictory events → fix vitals rules and regenerate.

T4 — Cross-modal consistency (the modalities tell the same story)

- **Purpose:** Confirm the structured record, notes, and vitals **agree** for each patient (R5): note diagnoses appear in the structured problem list; a note asserting a vitals event has a matching signal event; units and codes align.
- **How to run:** same command as T3; inspect the `consistency` section (defect list per patient).
- **Expected — PASS:** zero unresolved consistency defects across the cohort.
- **FAIL:** any consistency defect (e.g. note says "afebrile" but temperature curve shows a fever) → fix the shared-state mapping/templates and regenerate. This is the core multi-modal guarantee.

T5 — NLP-readiness of notes

- **Purpose:** Confirm synthesized notes are structurally usable for NLP testing — expected sections per note type, non-empty required slots, parseable text, and (where configured) that clinical entities/labels referenced by the state are actually present in the text.
- **How to run:** inspect the `notes` section of the consistency report (per-note-type structure and slot-fill checks).
- **Expected — PASS:** each note has its expected sections/slots filled and the state's key entities are findable in text.
- **FAIL:** empty/malformed notes or missing entities → fix templates/phrase banks and regenerate.

T6 — Synthetic-safety / no-real-PHI scan (release blocker)

- **Purpose:** Confirm the multi-modal set contains **no real PHI** — especially in free-text notes — and that any real reference data used stayed read-only/minimized (R6).
- **How to run:** same command as T4; inspect the `safety` section (synthetic-identifier allow-list check and real-name/MRN/date pattern scan).
- **Expected — PASS:** zero real-PHI pattern hits; all identifiers on the synthetic allow-list; reference handling documented read-only/minimized.

- **FAIL: any** hit is a hard block — do **not** release; investigate the template/state source, purge

the offending set, and regenerate. (This is a technical guardrail; the privacy/de-identification *determination* remains with privacy/compliance/legal.)

Release decision

Record the outcomes as **Multi-Modal Validation Report – Pilot – v1**. Release the set to AI/QA only when **T1–T6 all PASS** (Gate D). Any failure → rollback (discard the synthetic set) and regenerate. Attach the reports as evidence so every released set is reproducible and auditable.

Troubleshooting

Common failure modes when extending synthetic generation to multi-modal clinical data, and how to resolve them. All fixes stay inside the isolated non-production environment on synthetic data; rollback is always "discard the synthetic set and regenerate" because nothing production or real-PHI is touched.

The note and the vitals contradict each other (T4 consistency FAIL)

- **Symptom:** `crossmodal_consistency_check.py` reports defects such as a note stating "afebrile" while

the temperature channel shows a fever, or an asserted vitals event with no matching signal.

- **Cause:** The note and the vitals were **not** rendered from the same clinical state — the shared-state

contract (R2) is incomplete, or a template/vitals rule reads a different fact than the one generated.

- **Resolution:** Confirm both the note synthesizer and the vitals generator read the **same**

`clinical_state_facts`. Fix the template slot or the `channel_target` mapping so both key off the same fact, then regenerate. Never edit one modality's output by hand — always regenerate from state.

A note states a diagnosis that isn't in the structured problem list (T4 consistency FAIL)

- **Symptom:** Defect `note_diagnoses_in_structured_problem_list`.
- **Cause:** The note template injected a diagnosis code not present in `structured.problem_list`, or the

structured record was generated separately from the note.

- **Resolution:** Ensure the note's `stated_diagnoses` are derived from the structured record's problem

list (single source of truth), then regenerate. Add the diagnosis to the structured record *or* remove it from the note — do not leave them disagreeing.

Vitals out of physiologic range (T3 plausibility FAIL)

- **Symptom:** `plausibility.violations` lists values outside a channel's `plausible` band.

- **Cause:** A channel's target range (baseline or event) exceeds the configured plausible band, or the config's `normal`/`plausible` ranges are inconsistent (`normal` not inside `plausible`).
- **Resolution:** Run `scope_config_helper.py validate` — it flags ranges where `normal` isn't inside `plausible`. Correct the ranges/event targets in the config, then regenerate.

`scope_config_helper.py validate` reports FAIL

- **Symptom:** Missing sections, empty `note_types`/`consistency_rules`, or bad ranges.
- **Cause:** Config was hand-edited into an invalid state, or a required gate threshold was removed.
- **Resolution:** Re-run `init` to get a known-good skeleton, port your tuned values in, and re-validate. Keep the config in version control so you can diff against the last valid one.

A modality is missing for some patients (T1 coverage FAIL)

- **Symptom:** `fidelity_coverage_report.py` coverage < 100%; `missing` lists patients/modalities.
- **Cause:** The modality isn't in the config's `modalities`, or generation skipped it (e.g. no `note_types` defined but `notes` in scope).
- **Resolution:** Ensure every in-scope modality has its inputs (note types for notes, channels for vitals) and is listed in `modalities`, then regenerate the full cohort.

Fidelity shows "NOT-SCORED"

- **Symptom:** `fidelity_coverage_report.py` prints `fidelity=NOT-SCORED`.
- **Cause:** No `--reference` was supplied, so there is nothing to measure realism against (global requirement #6). This is **not** a pass — it means fidelity is unproven.
- **Resolution:** Provide read-only, minimized reference statistics (or agreed published reference distributions) via `--reference` and re-run. Realism claims are only valid against a stated reference.

Fidelity FAIL (distance over threshold)

- **Symptom:** A channel's normalized distance exceeds `fidelity_max_distance`.
- **Cause:** Generation parameters/ranges don't match the reference distribution, or the scenario mix skews a channel's mean.
- **Resolution:** Tune the channel's baseline/event ranges (or scenario weights) toward the reference, or, if the threshold is unrealistic for a synthetic pilot, agree a defensible threshold with the AI/QA and privacy stakeholders and record it. Re-validate.

Synthetic-safety scan hit — a real-PHI pattern (T6 FAIL — hard block)

- **Symptom:** `safety.hits` is non-empty (e.g. an SSN/MRN/date/email pattern, or a non-allow-listed identifier).
- **Cause:** A template or phrase bank contained real patient text, real data leaked into a slot value, or a patient used a non-synthetic identifier.
- **Resolution: Do not release.** Purge the offending set, trace the source (template, phrase bank, or state field) and remove the real content, then regenerate and re-scan until clean. Keep the synthetic-identifier allow-list (`SYN-#####`) current. This is a **technical guardrail** — the de-identification/disclosure-risk *determination* remains with privacy/compliance/legal.

Notes are empty or malformed (T5 NLP-readiness FAIL)

- **Symptom:** `notes.issues` lists empty required sections.
- **Cause:** A template slot mapped to a fact the state didn't set, so the section rendered blank.
- **Resolution:** Ensure every note-type section has a template phrase and a state fact to fill it; regenerate. Empty notes are unusable for NLP testing and must be fixed before release.

Vendor synthetic-data / AI framework behaves differently than documented here

- **Symptom:** A Synthea-style generator or commercial platform produces output that doesn't match the structure these tools expect.
- **Cause:** Vendor console internals and versions vary; this solution integrates at the pattern level.
- **Resolution: Verify version-specific behavior against the vendor's documentation** and adapt the mapping into the shared clinical state. Do not assume vendor internals — the shared-state contract is the integration boundary.

Best Practices

Durable practices for producing and maintaining multi-modal clinical synthetic data (structured + notes + vitals/time-series) that stays consistent, realistic, and safe.

Generate from one shared state — never author modalities independently

Cross-modal consistency is only achievable when the structured record, notes, and vitals all render from a **single per-patient clinical state** (the **Clinical State Contract – Pilot – v1**). The moment a note is written independently of the vitals, they drift and contradict. Treat the clinical state as the sole source of truth and derive every modality from it deterministically.

Make "consistent" measurable, and gate on it

Codify the consistency rules (note diagnoses ↔ structured problem list; asserted vitals events ↔ signal events; valid units/ranges) and the pass/fail thresholds **before** generating, and enforce them as a release

gate (Gate D). A dataset that isn't gated on consistency will quietly teach multi-modal models the wrong correlations.

Keep templates synthetic and version-controlled

Note templates are **slot-filling patterns with phrase banks**, never copies of real notes. Keep them in version control, review them with a clinical SME for plausibility, and label each slot with the clinical fact it renders. This keeps notes NLP-ready *and* free of real PHI.

Anchor vitals to physiology and to the note narrative

Define channel ranges, cadence, baselines, and event shapes from a documented reference. When a note asserts an event ("febrile to 39.2°C at hour 14"), the temperature channel must show it. Physiologic plausibility and note-alignment are both required — one without the other is a defect.

Validate against a reference, and state the reference

Fidelity claims are only as strong as the reference they're measured against (global §6). Always record which reference statistics or published distributions a set was validated against, so realism claims are defensible and reproducible.

Run the no-real-PHI scan on every set — free-text is the biggest risk surface

Notes are where real PHI is most likely to leak in. Make the synthetic-safety scan a **mandatory, release-blocking** step (T6), keep the synthetic-identifier allow-list current, and confirm any real reference data stayed read-only and minimized.

Pilot-first, expand only after the gate passes

Prove the full pipeline on a small cohort and pass every gate before widening the cohort or adding note types/vitals channels. Re-validate at each expansion step — new modalities and cohorts introduce new consistency and plausibility risks.

Keep the config the single driver; keep everything reproducible

Drive generation and validation from one validated **Multi-Modal Config – Pilot – v1**. Version the config, the templates, and the reports together so any released set can be regenerated and its gates re-audited. Reproducibility is what makes a synthetic set trustworthy to AI/QA teams.

Stay inside the boundary

Everything runs in the isolated non-production environment on synthetic data. The solution provides the technical **method and evidence**; HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, and disclosure-risk acceptance are **deferred to privacy/compliance/legal**. For vendor tools, integrate at the pattern level and verify version-specific behavior against the vendor's documentation.

Deliverables Summary

What you get with **Multi-Modal Clinical Synthetic Data (Structured + Clinical Notes + Vitals Time-Series)** — a self-guided package to extend synthetic generation beyond tables to consistent, NLP-ready notes and vitals/time-series for a single synthetic patient. Everything runs in an isolated non-production environment on **synthetic** data.

Documentation

• **Solution Overview** (`01-solution-overview.md`) — the multi-modal model (structured + notes + vitals), cross-modal consistency, the target model (FHIR/OMOP/native), benefits, limitations, and the Assess → Design → Generate/Build → Validate → Expand deployment model.

• **Requirements Guide** (`02-requirements.md`) — the solution-specific prerequisites (structured backbone, clinical-state contract, note templates, vitals rules, consistency rules, synthetic-safety control), each with how to verify and implement, on top of the six global requirements.

• **Implementation Guide** (`03-implementation.md`) — exact ordered steps, pilot-first, with named artifacts and explicit pass/fail gates (A–D) plus rollback.

• **Testing & Validation** (`05-testing.md`) — the T1–T6 procedures (coverage, fidelity, plausibility, cross-modal consistency, NLP-readiness, no-real-PHI safety), each with expected pass/fail.

• **Troubleshooting** (`06-troubleshooting.md`) — common multi-modal failure modes and resolutions.

• **Best Practices** (`07-best-practices.md`) — durable practices for consistent, realistic, safe multi-modal generation.

• **Executive Summary** (`executive-summary.md`) — the outcome, approach, and why it's safe.

The 5 Python tools

All are **safe by default** (dry-run where they write; no real PHI to shared locations), standard-library first (pandas/numpy optional), Python 3.10+, with `--help`:

1. `modality_scope_assess.py` — *modality-scope assessment.* Reads the AI/QA use cases and profiles which modalities (structured / notes / vitals) each needs, the target model, note types, and vitals channels. Produces the **Modality Scope – Pilot – v1** record.

1. `multimodal_generate.py` — *multi-modal generator.* Renders structured record + template/pattern clinical notes + vitals/time-series for each synthetic patient **from one shared clinical state**, so the modalities are consistent. Dry-run by default; `--write` opts in to the isolated harness. Produces **Multi-Modal Synthetic Set – Pilot – v1**.

1. `fidelity_coverage_report.py` — *fidelity & coverage report.* Measures modality coverage (every modality present per patient) and fidelity vs. a reference. Produces the fidelity/coverage portion of **Multi-Modal Validation Report – Pilot – v1**.

1. `crossmodal_consistency_check.py` — *cross-modal consistency & synthetic-safety validator.* Checks

the modalities tell the same story (diagnoses ↔ notes ↔ vitals events), physiologic plausibility, NLP-readiness of notes, and runs the no-real-PHI scan. Produces the consistency/safety portion of the validation report.

1. `scope_config_helper.py` — *scope/config helper.* Initializes and validates the single

Multi-Modal Config – Pilot – v1 (scope, note types, vitals channels/ranges, consistency rules, gate thresholds) that drives generation and validation.

Outcome

A **pilot-validated, reproducible multi-modal synthetic dataset** — structured records plus realistic notes and vitals/time-series, cross-modally consistent per synthetic patient — released to AI/QA teams with validation evidence, so they can test document-, signal-, and sequence-based models safely.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

crossmodal_consistency_check.py

fidelity_coverage_report.py

modality_scope_assess.py

multimodal_generate.py

scope_config_helper.py